

Robotic Systems in Manipulation - Phase II: Trajectory Execution

Rodrigo Sales Freire dos Santos (ist1119658)

Mikhail Mohamed Shueb (ist1104014)

June 2026

1 Introduction

The second part of the project extends the initial kinematic work developed in Phase I to a simple surgical trajectory execution problem. In this phase, the objective is not only to compute the position of the robot end-effector, but also to control the motion of a surgical tool while respecting a clinically relevant constraint.

The KUKA LBR MED robot is considered with a rigid tool fixed to the end-effector, representing a biopsy needle with length

$$L = 0.15 \text{ m.} \quad (1)$$

The needle must enter the body through a fixed trocar point, which represents the small incision used in minimally invasive surgery. The tooltip must then reach a sequence of target points inside the patient. During this motion, the main constraint is that the tool shaft cannot move sideways at the trocar. It can slide through the trocar and rotate around it, but the trocar point must remain on the needle axis at all times. This avoids lateral forces at the incision and reduces the risk of damaging the patient's skin or surrounding tissue.

This type of constraint is known as a Remote Center of Motion (RCM). It is called “remote” because the center of rotation is not located at a physical joint of the robot, but rather at a fixed virtual point in space along the surgical tool axis. In practical terms, the robot motion is restricted so that the needle can only perform insertion/retraction along its own axis and angular motion around the trocar point.

In our implementation, the trocar position was defined as

$$\mathbf{c}_r = \begin{bmatrix} -0.50 \\ 0.00 \\ 0.40 \end{bmatrix} \text{ m.} \quad (2)$$

The trajectory was simulated in `Surgical_RCM_Sim.slx`, using the Direct Kinematics and Jacobian functions already obtained in the first phase of the project. These functions allow the robot configuration to be related to the position and orientation of the end-effector, while the Jacobian is used to describe how joint velocities affect the motion of the tooltip and the tool axis. Therefore, the Phase I kinematic model becomes the basis for controlling the robot in a surgical scenario with an RCM constraint.

Overall, this problem represents a simplified version of robot-assisted minimally invasive surgery, where the robot must accurately guide the tooltip to internal targets while keeping the surgical entry point fixed and safe.

2 Setup

2.1 Target Points and Trajectory Planning

The selected targets were placed inside the reachable workspace of the manipulator and below the trocar. This makes the setup consistent with the surgical situation: the robot flange stays above the trocar and the tooltip moves inside the body.

Table 1: Selected target points and time segments for the tooltip trajectory.

Point	Time segment [s]	Coordinates (x, y, z) [m]	Description
m_0	0–5	$(-0.50, 0.00, 0.30)$	Initial internal target point
m_1	5–10	$(-0.50, 0.08, 0.28)$	Second internal target point
m_2	10–15	$(-0.52, 0.04, 0.27)$	Deepest insertion target
m_3	15–20	$(-0.46, -0.04, 0.32)$	Highest and shallowest target
m_4	20–25	$(-0.50, -0.08, 0.28)$	Fifth internal target point
m_5	25–30	$(-0.55, 0.00, 0.30)$	Final target and hold

For each segment, a cubic time law with duration

$$T = 5 \text{ s} \quad (3)$$

was used. The velocity is zero at the beginning and at the end of every segment. This avoids sudden jumps when the tooltip reaches a target.

For a segment starting at position $\mathbf{p}_i$ and ending at position $\mathbf{p}_f$, the normalized cubic interpolation variable is given by

$$s(\tau) = 3 \left(\frac{\tau}{T} \right)^2 - 2 \left(\frac{\tau}{T} \right)^3, \quad 0 \leq \tau \leq T, \quad (4)$$

where τ is the local time inside the current segment.

The derivative of the cubic time law is

$$\dot{s}(\tau) = \frac{6\tau}{T^2} - \frac{6\tau^2}{T^3}. \quad (5)$$

Therefore, the desired tooltip position is

$$\mathbf{p}_d(\tau) = \mathbf{p}_i + s(\tau)(\mathbf{p}_f - \mathbf{p}_i), \quad (6)$$

and the desired tooltip velocity is

$$\mathbf{v}_d(\tau) = \dot{s}(\tau)(\mathbf{p}_f - \mathbf{p}_i). \quad (7)$$

The same interpolation idea can be used to define the desired orientation of the tool during the trajectory, ensuring that the needle axis remains compatible with the trocar constraint.

2.2 Initial Configuration

To start the simulation, a valid initial posture is required. The chosen initial tooltip position was

$$\mathbf{m}_0 = \begin{bmatrix} -0.50 \\ 0.00 \\ 0.30 \end{bmatrix}. \quad (8)$$

The trocar is located 10 cm above this initial tooltip position. In the implementation, the tool axis is taken as the third column of the end-effector rotation matrix. At the initial pose, this axis points from the flange down toward the tooltip:

$$\mathbf{z}_e = \begin{bmatrix} 0 \\ 0 \\ -1 \end{bmatrix}. \quad (9)$$

Since the tool has length $L = 0.15$ m, the end-effector position must start at

$$\mathbf{p}_e = \mathbf{m}_0 - L\mathbf{z}_e. \quad (10)$$

This places the flange above the trocar, while the tooltip starts at $\mathbf{m}_0$. The analytical inverse kinematics solver from Phase I was then used to obtain the initial joint vector:

$$\mathbf{q}_0 = [q_{1,0} \ q_{2,0} \ q_{3,0} \ q_{4,0} \ q_{5,0} \ q_{6,0} \ q_{7,0}]^T. \quad (11)$$

This initial configuration was used in the Simulink model.

3 Operation Constraint

As mentioned above, the trocar imposes a surgical operation constraint. This means that all needle movements must keep the tool shaft passing through the trocar point.

At each instant, the direct kinematics gives the end-effector position $\mathbf{p}_e$, the rotation matrix $\mathbf{R}_e$, and the tool axis $\mathbf{z}_e$, which is the third column of $\mathbf{R}_e$. In the controller implementation, the tooltip position is computed as

$$\mathbf{p}_{tool} = \mathbf{p}_e + L\mathbf{z}_e. \quad (12)$$

To check the trocar constraint, the trocar point $\mathbf{c}_r$ is projected onto the tool axis. The insertion distance measured from the flange along the tool axis is

$$\lambda = \mathbf{z}_e^T (\mathbf{c}_r - \mathbf{p}_e). \quad (13)$$

The closest point on the tool shaft to the trocar is then

$$\mathbf{p}_c = \mathbf{p}_e + \lambda\mathbf{z}_e. \quad (14)$$

Using this point, the trocar error is

$$\mathbf{e}_{RCM} = \mathbf{c}_r - \mathbf{p}_c. \quad (15)$$

If this error is zero, the needle axis passes exactly through the trocar.

However, the controller should not correct every possible motion of the tool. Motion along the needle direction is allowed, because that is simply inserting or withdrawing the

tool through the trocar. The forbidden motion is only the lateral motion, perpendicular to the needle axis, because that would push the trocar sideways.

For this reason, the RCM Jacobian is projected onto the plane perpendicular to the tool axis. The projection matrix is defined as

$$\mathbf{P}_\perp = \mathbf{I}_3 - \mathbf{z}_e \mathbf{z}_e^T. \quad (16)$$

Thus, only the lateral part of the trocar error is considered:

$$\mathbf{e}_\perp = \mathbf{P}_\perp \mathbf{e}_{RCM}. \quad (17)$$

The projected RCM Jacobian is

$$\mathbf{J}_{RCM,\perp} = \mathbf{P}_\perp \mathbf{J}_{RCM}. \quad (18)$$

This makes the controller correct only the sideways error at the trocar, while still allowing the needle to slide in and out normally.

4 Simulation

4.1 CLIK Controller

The controller uses the same Closed-Loop Inverse Kinematics (CLIK) idea from Phase I. However, in this phase, the controller has two tasks with different priorities:

1. Keep the tool shaft passing through the trocar point.
2. Move the tooltip to the desired target position.

The RCM task is treated as the highest priority task. The lateral trocar error is corrected with

$$\mathbf{v}_{RCM}^* = \mathbf{K}_{RCM} \mathbf{e}_\perp, \quad (19)$$

where $\mathbf{K}_{RCM}$ is the RCM correction gain. The corresponding joint velocity contribution is

$$\dot{\mathbf{q}}_{RCM} = \mathbf{J}_{RCM,\perp}^\dagger \mathbf{v}_{RCM}^*. \quad (20)$$

This first velocity reduces the lateral error at the trocar. The tooltip tracking task is then added without disturbing the RCM task. The tooltip position error is

$$\mathbf{e}_{tool} = \mathbf{p}_d - \mathbf{p}_{tool}. \quad (21)$$

The desired tooltip velocity with feedback correction is

$$\mathbf{v}_{tool}^* = \mathbf{v}_d + \mathbf{K}_{tool} \mathbf{e}_{tool}, \quad (22)$$

where $\mathbf{K}_{tool}$ is a positive gain matrix.

Since part of the tooltip motion is already produced by $\dot{\mathbf{q}}_{RCM}$, the remaining tooltip velocity to be generated is

$$\mathbf{v}_{tool,rem} = \mathbf{v}_{tool}^* - \mathbf{J}_{tool} \dot{\mathbf{q}}_{RCM}. \quad (23)$$

To preserve the RCM task, the tooltip correction is projected into the null space of the RCM Jacobian. The null-space projector is

$$\mathbf{N}_{RCM} = \mathbf{I}_7 - \mathbf{J}_{RCM,\perp}^\dagger \mathbf{J}_{RCM,\perp}. \quad (24)$$

The projected tooltip Jacobian is

$$\mathbf{J}_{tool,N} = \mathbf{J}_{tool} \mathbf{N}_{RCM}. \quad (25)$$

The secondary joint velocity is then computed with a damped pseudoinverse:

$$\dot{\mathbf{q}}_{tool} = \mathbf{J}_{tool,N}^\# \mathbf{v}_{tool,rem}, \quad (26)$$

where $\mathbf{J}_{tool,N}^\#$ represents the damped pseudoinverse used to avoid large joint velocities near singular configurations.

Finally, the joint velocity command sent to the integrator is

$$\dot{\mathbf{q}} = \dot{\mathbf{q}}_{RCM} + \dot{\mathbf{q}}_{tool}. \quad (27)$$

This task-priority formulation makes the controller correct the trocar constraint first. The tooltip trajectory is followed as a secondary task, using only joint motion that does not intentionally violate the RCM constraint.

4.2 Simulink Model

The simulation was implemented in `Surgical_RCM_Sim.slx`. The model receives the initial joint configuration $\mathbf{q}_0$ and computes, at each simulation step, the direct kinematics, the tooltip position, the RCM error, and the joint velocity command generated by the CLIK controller.

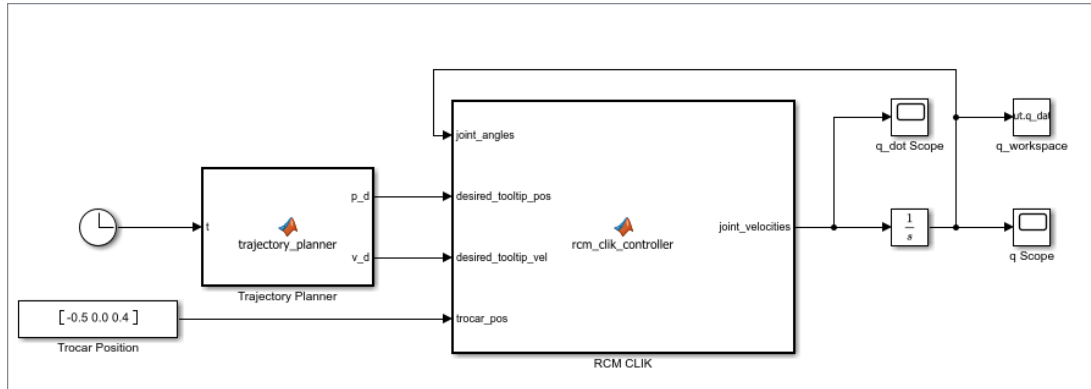


Figure 1: Simulink model

The model is built around a closed feedback loop: the current joint angles are used by the controller to compute the robot pose, and the resulting joint velocities are integrated to update the robot configuration over time.

The trajectory planner block receives the simulation time and outputs the desired tooltip position and velocity for the current segment of the path. These references are sent to the RCM CLIK controller, together with the trocar position and the current joint angles. Inside the controller, the KUKA forward kinematics are used to obtain the flange position and orientation. The flange z-axis defines the direction of the needle, allowing

the controller to compute the tooltip position and the distance between the trocar and the needle shaft.

The controller gives priority to the trocar constraint. First, it calculates a joint velocity contribution that reduces the RCM error. Then, the tooltip tracking motion is projected into the null space of the RCM task, so that following the target trajectory does not intentionally disturb the trocar constraint. The final joint velocity command is integrated by the Simulink integrator block, producing the joint positions used in the next simulation step. This structure allows the Phase II model to simulate the continuous closed-loop behavior of the surgical robot during the complete 30 s trajectory.

4.3 Controller Gain Selection

The controller gains were selected to balance trajectory tracking accuracy and numerical stability. A higher tooltip gain makes the robot correct position errors faster, while the RCM gain increases the correction of lateral trocar deviations. However, excessively high gains may generate large joint velocities or oscillations.

A damped pseudoinverse was used instead of a standard pseudoinverse to improve numerical stability near singular configurations. The damping factor prevents excessive joint velocities, at the cost of a small transient tracking error.

In the final simulation, the tooltip gain was set to $K_{tool} = 20$ and the damping factor was set to $\lambda_{damp} = 0.01$.

5 Results

After running the simulation, the script `validate_rcm_simulation.m` was used to reconstruct the end-effector and tooltip positions from the saved joint data. For each time sample, the script recomputed the distance between the trocar and the closest point on the tool shaft.

The 3D result is shown in Figure 2. The blue line represents the tooltip path, the green points represent the target positions, the cyan marker represents the trocar, and the orange lines show different sampled positions of the needle shaft.

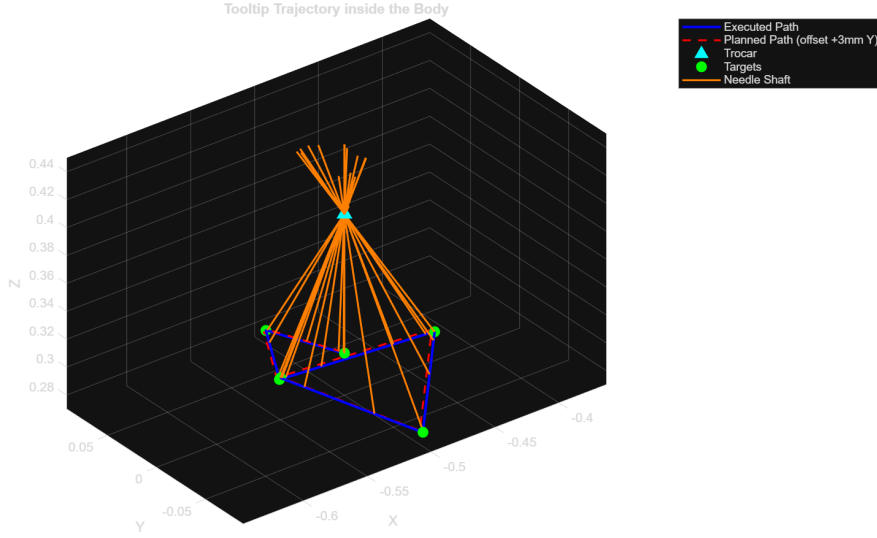


Figure 2: 3D visualization of the tooltip trajectory and sampled needle shaft positions.

The figure shows that the tooltip follows the planned internal path while the needle shaft stays aligned with the trocar. The shaft rotates around the trocar and changes insertion depth, but it does not drift freely through the body wall.

The trocar error over time is shown in Figure 3.

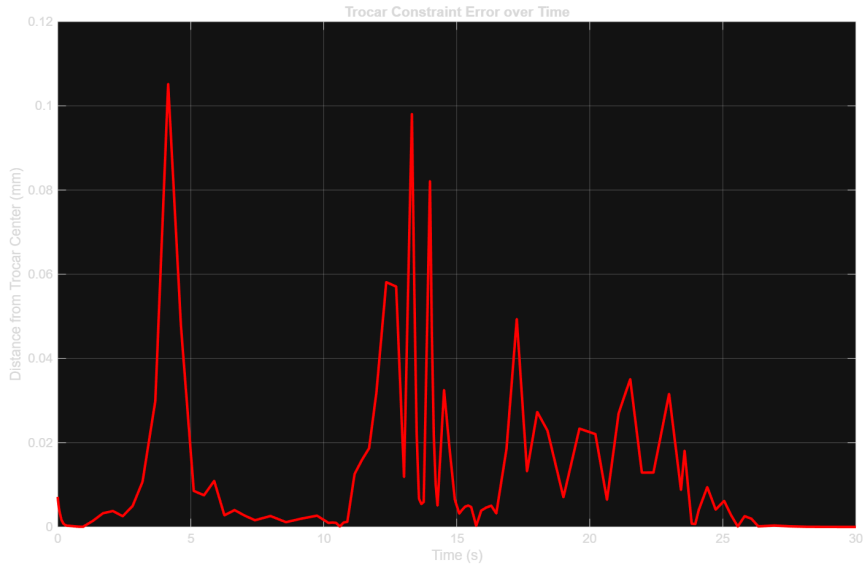


Figure 3: Trocar error over time during the trajectory execution.

The maximum error stays at about

$$e_{RCM,max} \approx 0.105 \text{ mm.} \quad (28)$$

Most of the time, the error is lower than this value, with small peaks during the trajectory transitions. Since the error remains well below 1 mm, the constraint of the problem is respected for the simulated trajectory.

Table 2: RCM error for each trajectory segment.

Segment	Time interval [s]	Target point	Maximum RCM error [mm]
1	0–5	$m_0 \rightarrow m_1$	0.105213
2	5–10	$m_1 \rightarrow m_2$	0.010926
3	10–15	$m_2 \rightarrow m_3$	0.098094
4	15–20	$m_3 \rightarrow m_4$	0.049387
5	20–25	$m_4 \rightarrow m_5$	0.035113
6	25–30	m_5 hold	0.006190

5.1 Error Margin Statistics

In addition to the maximum RCM error per trajectory segment, the tooltip tracking error was also evaluated. This error measures the distance between the desired tooltip position and the executed tooltip position during the simulation.

Table 3 shows the RCM error and tooltip tracking error at each target point. The results confirm that both the trocar constraint and the tooltip trajectory were tracked with sub-millimeter accuracy.

Table 3: Error margin at each target point.

Target point	Time [s]	Trocar RCM error [mm]	Tooltip tracking error [mm]
m_0 (Start)	0	0.007143	0.011697
m_1	5	0.008550	0.065520
m_2	10	0.000953	0.129795
m_3	15	0.003167	0.049255
m_4	20	0.022055	0.141048
m_5 (Start)	25	0.006190	0.036866
m_5 (End)	30	0.000000	0.000000

The average and maximum errors during each transition are shown in Table 4. The largest tooltip tracking error was approximately 0.186 mm, while the largest RCM error was approximately 0.105 mm. These values are very small compared with the scale of the robot motion and indicate that the controller maintained the trocar constraint accurately throughout the simulated trajectory.

Table 4: Average and maximum tracking errors during trajectory transitions.

Transition	Avg RCM error [mm]	Max RCM error [mm]	Avg tooltip error [mm]	Max tooltip error [mm]
$m_0 \rightarrow m_1$	0.009865	0.105213	0.020335	0.185847
$m_1 \rightarrow m_2$	0.003946	0.010926	0.028859	0.129795
$m_2 \rightarrow m_3$	0.022614	0.098094	0.038649	0.129795
$m_3 \rightarrow m_4$	0.012369	0.049387	0.034262	0.141048
$m_4 \rightarrow m_5$	0.013092	0.035113	0.034027	0.141048
m_5 hold	0.001101	0.006190	0.007714	0.036866

A small initial error is visible at the beginning of the simulation. This is due to the rounded values used for the initial joint configuration in the Simulink model. Since the

initial joint values are inserted with limited numerical precision, the reconstructed tooltip and trocar positions are not exactly equal to the desired values at $t = 0$. However, this error is very small and is corrected immediately by the closed-loop controller.

6 Comparative Analysis: CLIK vs. Analytical Inverse Kinematics

To evaluate the trajectory execution strategy, the Closed-Loop Inverse Kinematics (CLIK) controller can be compared with the analytical inverse kinematics method developed in Phase I. Both approaches can be used to generate robot motion, but they behave differently when applied to continuous trajectory tracking and RCM constraint enforcement.

6.1 Path Planning and Computational Complexity

Analytical inverse kinematics provides a direct, closed-form mapping from a desired Cartesian pose to a joint configuration. This is useful for computing isolated target configurations. However, for a continuous trajectory, the analytical solver must be evaluated at every intermediate point. In addition, because redundant robots can have multiple valid configurations for the same tooltip pose, the selected inverse kinematics branch must remain consistent throughout the motion. Otherwise, discontinuities or sudden joint changes may occur.

CLIK, on the other hand, works at the velocity level. Instead of solving a complete inverse kinematics problem for each target pose, it computes the joint velocities required to reduce the tooltip and RCM errors. These velocities are then integrated over time. This tends to produce smooth joint trajectories and makes the path planning process part of the control loop itself.

6.2 Path Tracking and RCM Constraint Enforcement

In the analytical inverse kinematics approach, the RCM constraint can be enforced geometrically by defining the desired flange position and orientation with respect to the trocar. Under ideal conditions, this can produce zero theoretical RCM error. However, this approach is open-loop: once the joint configuration is computed, the method does not actively correct errors caused by numerical drift, small modeling inaccuracies, or disturbances.

The trajectory obtained under the analytical IK approach is shown in Figure 4. In this ideal simulation case, the needle shaft remains aligned with the trocar because the end-effector pose is directly constructed from the desired tooltip and trocar geometry.

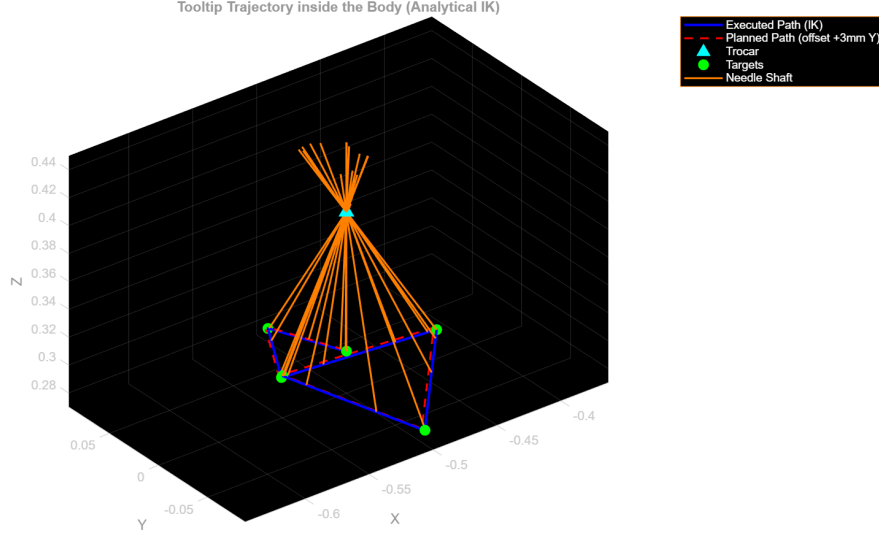


Figure 4: 3D simulation of the tooltip trajectory and needle shaft passing through the trocar under open-loop analytical IK control in ideal conditions.

In the CLIK approach, the trocar constraint is treated as an active feedback task. At each simulation step, the controller recomputes the distance between the trocar and the needle shaft, and then generates a correction velocity to reduce this error. This makes the method more robust for trajectory execution because the controller continuously reacts to deviations during the motion.

6.3 Error Characteristics

Under ideal, noise-free simulation conditions, analytical inverse kinematics can produce zero theoretical tooltip and RCM error if the desired pose is exactly reachable and the robot executes the computed joint values perfectly. This is shown in Figure 5, where the RCM error remains flat at zero for the analytical IK case.

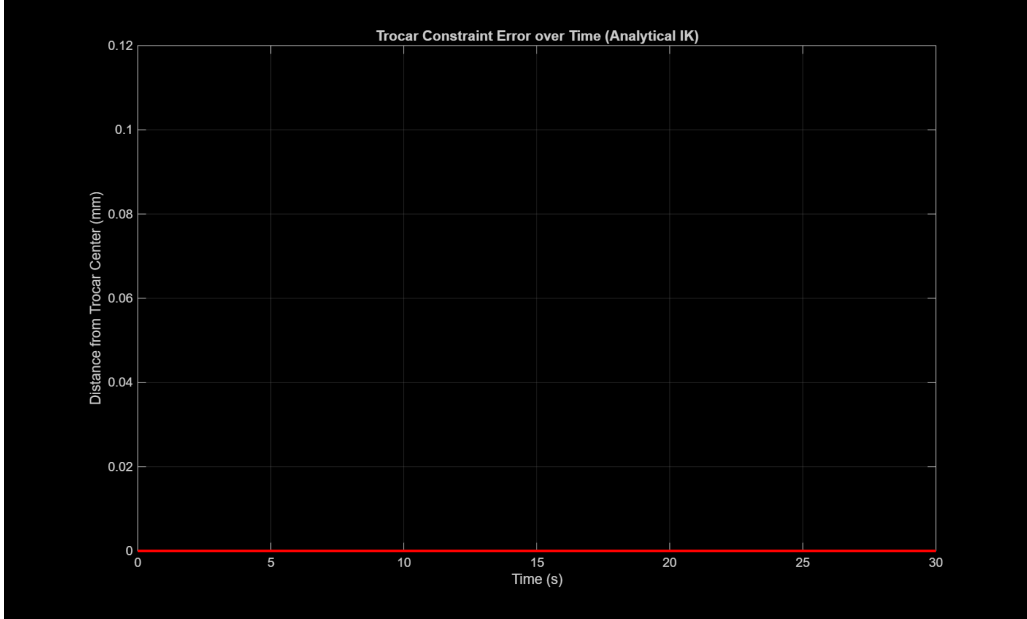


Figure 5: Trocar constraint error magnitude over time under analytical IK control. In ideal simulation conditions, the RCM error remains equal to zero.

In contrast, CLIK may introduce very small transient errors because it depends on numerical integration, controller gains, and pseudoinverse damping. In this simulation, the CLIK controller kept the maximum tooltip tracking error below 0.19 mm and the maximum RCM error close to 0.105 mm. These errors occur mainly during trajectory transitions, when the desired tooltip velocity changes direction. During the final hold phase, the error converges to zero.

6.4 Singularity Handling and Joint Smoothness

Another important difference is how the methods behave near singular configurations. Analytical inverse kinematics may become unstable or fail when the robot approaches configurations where the closed-form equations are ill-conditioned. In these cases, small Cartesian changes can produce very large joint variations.

The CLIK controller uses a damped pseudoinverse, which improves numerical stability near singularities. The damping factor acts as a soft limitation on excessive joint velocities. This may introduce a small temporary tracking error, but it avoids abrupt or unrealistic joint commands.

For the trajectory used in this report, both analytical IK and CLIK can generate very similar joint profiles because the path does not force a major branch change. This is shown in Figure 6, where the selected joint trajectories almost overlap. Therefore, the plot does not indicate a failure of analytical IK in this specific case; instead, it shows that both methods produce similar joint motion for this trajectory. However, CLIK remains more appropriate for closed-loop trajectory execution because it naturally integrates joint velocities over time and actively corrects tracking and RCM errors.

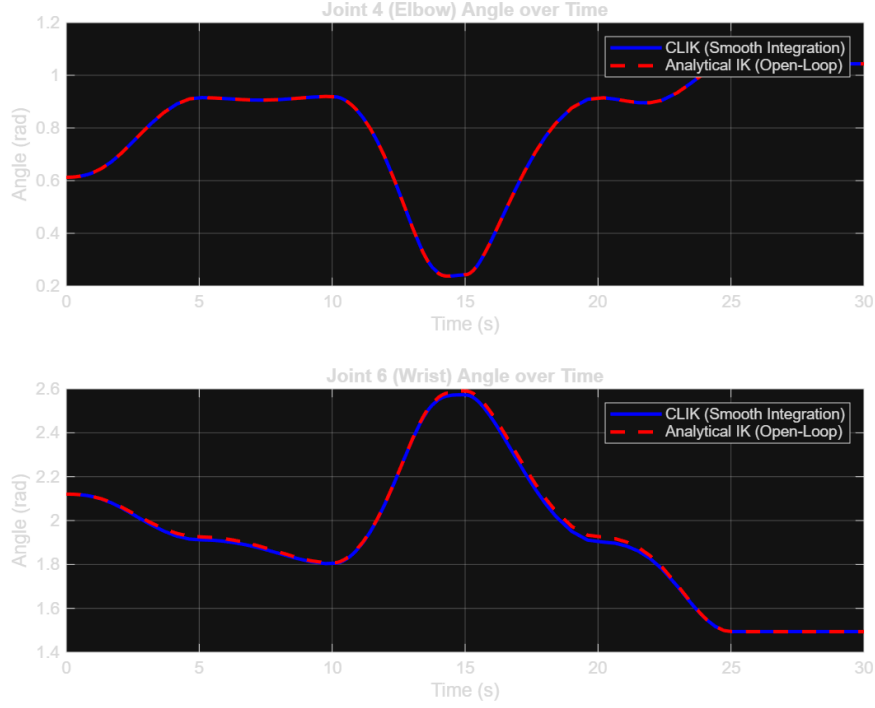


Figure 6: Comparison of selected joint trajectories for CLIK and analytical IK. For this trajectory, both methods produce almost identical smooth joint profiles, indicating that no major branch switching occurred.

Table 5: Comparison of CLIK and analytical inverse kinematics.

Metric	CLIK	Analytical IK
Mathematical basis	Velocity-level integration using the Jacobian	Closed-form geometric equations
RCM enforcement	Active feedback correction during motion	Geometric pose construction under ideal conditions
Robustness	More robust to numerical drift and small disturbances	Sensitive to open-loop errors and branch selection
Singularity handling	Damped pseudoinverse gives smoother degradation	May fail or generate large joint changes near singularities
Joint trajectory	Smooth due to velocity-level integration	Smooth in this test, but may suffer from branch-switching discontinuities
Tracking accuracy	Sub-millimeter transient errors	Theoretically exact under ideal execution

7 Conclusion and Results Analysis

In this phase, the CLIK controller was used to execute a surgical tooltip trajectory while keeping the RCM constraint at the trocar. The simulation lasted 30 seconds and followed six target points, from m_0 to m_5 . The same trajectory was also tested with the analytical inverse kinematics method from Phase I, so that both approaches could be compared.

From the simulation results, the main conclusions are:

1. **Tracking and RCM error:** With the chosen gains ($K_{tool} = 20$ and $\lambda_{damp} = 0.01$), the controller kept the RCM error small during the whole trajectory. The maximum RCM error was about 0.105 mm, and the average error stayed below 0.02 mm. The tooltip also followed the planned path with sub- millimeter error, with a maximum tracking error of about 0.186 mm.
2. **CLIK compared with analytical IK:** The analytical IK method gave almost zero error in the ideal simulation, because the desired pose was built directly from the trocar and tooltip geometry. However, this method is open-loop, so it does not correct errors once the joint values are computed. The CLIK controller is more useful for trajectory execution because it keeps checking the tooltip and RCM errors during the motion and corrects them at each step.
3. **Joint motion:** For this trajectory, both methods produced similar joint profiles, and no major branch switching appeared. Even so, CLIK is safer for continuous motion because it works with joint velocities instead of solving each target independently. This helps avoid sudden changes in the joint angles when the robot has more than one possible IK solution.
4. **Singularities:** The damped pseudoinverse used in the CLIK controller helps reduce large joint velocities near singular configurations. This can introduce a small tracking error, but it makes the controller more stable than using an undamped inverse.

Overall, the results show that the Phase II controller was able to follow the selected surgical path while keeping the trocar constraint almost fixed. The analytical IK solution is useful as a reference, but the CLIK approach is more appropriate for closed- loop trajectory execution because it can correct errors during the simulation.